# SQL JOINs

## First do it by hand. Then write it in SQL.

This guide shows you how to combine two tables.
First, we do it with our hands (manual).
Then, we see how SQL does the same thing.

### These are our two tables:

| emp_id | first_name | dept_id |
|---|---|---|
| 100 | Steven | 90 |
| 101 | Neena | 90 |
| 102 | Lex | NULL |
| 107 | Diana | 10 |
| 178 | Kimberely | 10 |

| dept_id | dept_name |
|---|---|
| 10 | Administration |
| 20 | Marketing |
| 90 | Executive |
| 110 | Accounting |

We want to see employee name + department name together.
But they are in different tables!
Both tables have a column called dept_id. This is the connection between them.

# Part 1: What Do We Want?

We have two tables. EMPLOYEES has names. DEPARTMENTS has department names. We want one result that shows: name + department name. On one line. For each person.

Think of it like a phone book problem. You have a list of people. You have another list of phone numbers. You want to make one new list that shows: person name + phone number. To do this, you match the person from list 1 with the phone number from list 2 using the person's name (it is in both lists).

The dept_id column works the same way. It is in both tables. We use it to match a person with their department.

## What we want the result to look like:

| first_name | dept_name |
|---|---|
| Steven | Executive |
| Neena | Executive |
| Diana | Administration |
| Kimberely | Administration |

How do we get this result? Let us first do it by hand (manual).

# Part 2: INNER JOIN

Rule for INNER: We only keep rows that match in BOTH tables. If a row does not have a match, we throw it away.

## STEP A: HOW WE DO IT BY HAND (MANUAL)

### Manual Step 1: Take Steven's row

| emp_id | first_name | dept_id |
|---|---|---|
| 100 | Steven | 90 |
| 101 | Neena | 90 |
| 102 | Lex | NULL |
| 107 | Diana | 10 |
| 178 | Kimberely | 10 |

| dept_id | dept_name |
|---|---|
| 10 | Administration |
| 20 | Marketing |
| 90 | Executive |
| 110 | Accounting |

Steven has dept_id = 90. We look at the departments table. We go down the dept_id column. Is there a 90? Yes! Row 3 has dept_id = 90 and dept_name = Executive. We found a match! We write on paper: Steven | Executive.

### Manual Step 2: Take Neena's row

| emp_id | first_name | dept_id |
|---|---|---|
| 100 | Steven | 90 |
| 101 | Neena | 90 |
| 102 | Lex | NULL |
| 107 | Diana | 10 |
| 178 | Kimberely | 10 |

| dept_id | dept_name |
|---|---|
| 10 | Administration |
| 20 | Marketing |
| 90 | Executive |
| 110 | Accounting |

Neena has dept_id = 90. We look at departments again. dept_id = 90 is there. dept_name = Executive. Match! We write on paper: Neena | Executive.

### Manual Step 3: Take Lex's row

| emp_id | first_name | dept_id |
|---|---|---|
| 100 | Steven | 90 |
| 101 | Neena | 90 |
| 102 | Lex | NULL |
| 107 | Diana | 10 |
| 178 | Kimberely | 10 |

| dept_id | dept_name |
|---|---|
| 10 | Administration |
| 20 | Marketing |
| 90 | Executive |
| 110 | Accounting |

Lex has dept_id = NULL. NULL means empty. No value. We look at departments. Can we find NULL in the dept_id column? No. NULL can not match any number. There is no match. We do NOT write Lex on paper. We skip him.

### Manual Step 4: Take Diana's row

| emp_id | first_name | dept_id |
|---|---|---|
| 100 | Steven | 90 |
| 101 | Neena | 90 |
| 102 | Lex | NULL |
| 107 | Diana | 10 |
| 178 | Kimberely | 10 |

| dept_id | dept_name |
|---|---|
| 10 | Administration |
| 20 | Marketing |
| 90 | Executive |
| 110 | Accounting |

Diana has dept_id = 10. We look at departments. dept_id = 10 is there. dept_name = Administration. Match! We write on paper: Diana | Administration.

### Manual Step 5: Take Kimberely's row

| emp_id | first_name | dept_id |
|---|---|---|
| 100 | Steven | 90 |
| 101 | Neena | 90 |
| 102 | Lex | NULL |
| 107 | Diana | 10 |
| 178 | Kimberely | 10 |

| dept_id | dept_name |
|---|---|
| 10 | Administration |
| 20 | Marketing |
| 90 | Executive |
| 110 | Accounting |

Kimberely has dept_id = 10. We look at departments. dept_id = 10 is there. dept_name = Administration. Match! We write on paper: Kimberely | Administration.

### Manual Step 6: Check departments with no employee

Marketing has dept_id = 20. Is there an employee with dept_id = 20? No. We skip it.
Accounting has dept_id = 110. Is there an employee with dept_id = 110? No. We skip it.
We finished all rows. Our paper has 4 lines.

## Our paper (manual result):

| first_name | dept_name |
|---|---|
| Steven | Executive |
| Neena | Executive |
| Diana | Administration |
| Kimberely | Administration |

## STEP B: HOW WE WRITE THIS IN SQL

Now let us see: what we did by hand, line by line. And what SQL code does the same thing.

### The SQL query:

```
SELECT e.first_name, d.dept_name
FROM employees e
INNER JOIN departments d
ON e.dept_id = d.dept_id;
```

| What we did by hand | SQL code that does the same |
|---|---|
| We took the EMPLOYEES table | FROM employees e |
| We also took the DEPARTMENTS table | INNER JOIN departments d |
| We matched dept_id in both tables | ON e.dept_id = d.dept_id |
| We wrote name + dept name on paper | SELECT e.first_name, d.dept_name |
| We skipped rows with no match | INNER JOIN (the word INNER does this) |
| "e" and "d" are short names for tables | e = employees, d = departments |

### Line by line:

FROM employees e = start with employees. Call it "e" (short name).
INNER JOIN departments d = also take departments. Call it "d". The word INNER means: skip rows with no match.
ON e.dept_id = d.dept_id = match rows where dept_id in employees is the same as dept_id in departments. This is like: when we looked at the dept_id in employees and searched for it in departments.
SELECT e.first_name, d.dept_name = show only name and department name in result. Not all columns. Only what we want.

What about "e." and "d." ?
When we use two tables, some columns have the same name (dept_id). If we write just "dept_id", SQL does not know: which table? So we write "e.dept_id" = dept_id from employees. "d.dept_id" = dept_id from departments. The letter before the dot tells SQL: this column is from that table.

# Part 3: LEFT JOIN

Rule for LEFT: We keep ALL rows from the left table (employees). If a row has no match, we still keep it. We write "-" (empty) for the department name.

## STEP A: HOW WE DO IT BY HAND (MANUAL)

Steps 1, 2, 4, 5 are the same as Part 2 (Steven, Neena, Diana, Kimberely all match). The difference is Step 3 (Lex). Let us look at that step.

### Manual Step 3 (Lex) with LEFT rule:

| emp_id | first_name | dept_id |
|---|---|---|
| 100 | Steven | 90 |
| 101 | Neena | 90 |
| 102 | Lex | NULL |
| 107 | Diana | 10 |
| 178 | Kimberely | 10 |

| dept_id | dept_name |
|---|---|
| 10 | Administration |
| 20 | Marketing |
| 90 | Executive |
| 110 | Accounting |

Lex has dept_id = NULL. We look at departments. No match. But this time, the rule is LEFT. LEFT means: keep ALL employees. So we do NOT skip Lex. We keep him on paper. But what do we write for his department? There is no department. We write "-" (empty, no department).
We write: Lex | -

### Our paper (manual result):

| first_name | dept_name |
|---|---|
| Steven | Executive |
| Neena | Executive |
| Lex | - (no dept) |
| Diana | Administration |
| Kimberely | Administration |

5 lines now. Lex is here with "-" for department. Marketing and Accounting are still not here (they are in the other table).

## STEP B: HOW WE WRITE THIS IN SQL

```
SELECT e.first_name, d.dept_name
FROM employees e
LEFT JOIN departments d
ON e.dept_id = d.dept_id;
```

What changed? Only one word. We changed INNER to LEFT.
INNER JOIN = skip rows with no match.
LEFT JOIN = keep all left table rows. Write NULL if no match.
In our manual work, LEFT means: we kept Lex even though he had no department. In SQL, the word LEFT does the same. The "-" we wrote by hand is NULL in SQL.

| What we did by hand | SQL code that does the same |
|---|---|
| We kept ALL employees (even no match) | LEFT JOIN (the word LEFT does this) |
| We wrote "-" for no department | SQL writes NULL (same meaning) |
| Everything else is the same | Same FROM, same ON, same SELECT |

# Part 4: RIGHT JOIN

Rule for RIGHT: We keep ALL rows from the right table (departments). This time, we start reading from the departments table, not employees.

## STEP A: HOW WE DO IT BY HAND (MANUAL)

This time, we change our plan. Before, we started from employees and looked at departments. Now, we start from departments and look at employees.

### Manual Step 1: Take Administration (dept_id = 10)

| emp_id | first_name | dept_id |
|---|---|---|
| 100 | Steven | 90 |
| 101 | Neena | 90 |
| 102 | Lex | NULL |
| 107 | Diana | 10 |
| 178 | Kimberely | 10 |

| dept_id | dept_name |
|---|---|
| 10 | Administration |
| 20 | Marketing |
| 90 | Executive |
| 110 | Accounting |

Administration has dept_id = 10. We look at employees. Is there an employee with dept_id = 10? Yes!
Diana has 10. Kimberely has 10. Two matches!
We write: Diana | Administration
We write: Kimberely | Administration

### Manual Step 2: Take Marketing (dept_id = 20)

| emp_id | first_name | dept_id |
|---|---|---|
| 100 | Steven | 90 |
| 101 | Neena | 90 |
| 102 | Lex | NULL |
| 107 | Diana | 10 |
| 178 | Kimberely | 10 |

| dept_id | dept_name |
|---|---|
| 10 | Administration |
| 20 | Marketing |
| 90 | Executive |
| 110 | Accounting |

Marketing has dept_id = 20. We look at employees. Is there an employee with dept_id = 20? No. No employee works in Marketing. But the rule is RIGHT. RIGHT means: keep ALL departments. So we keep Marketing. We write "-" for the employee name.
We write: - | Marketing

(The same happens for Accounting, dept_id = 110: - | Accounting)

## Our paper (manual result):

| first_name | dept_name |
|---|---|
| Diana | Administration |
| Kimberely | Administration |
| - (no emp) | Marketing |
| Steven | Executive |
| Neena | Executive |
| - (no emp) | Accounting |

## STEP B: HOW WE WRITE THIS IN SQL

```
SELECT e.first_name, d.dept_name
FROM employees e
RIGHT JOIN departments d
ON e.dept_id = d.dept_id;
```

What changed? We changed INNER to RIGHT.
RIGHT JOIN = keep all right table rows (departments). Write NULL if no match.
In our manual work, RIGHT means: we started from departments and kept them all. Marketing and Accounting stayed on our paper with "-". In SQL, RIGHT JOIN does the same.

| What we did by hand | SQL code that does the same |
|---|---|
| We started from departments table | SQL reads from the RIGHT table first |
| We kept ALL departments (even no match) | RIGHT JOIN (the word RIGHT does this) |
| We wrote "-" for no employee | SQL writes NULL (same meaning) |
| Lex was skipped (left table) | RIGHT protects right table, not left |

# Part 5: FULL OUTER JOIN

Rule for FULL OUTER: Keep EVERYTHING. From both tables. No row is removed. If no match, write "-" on the empty side.

## STEP A: HOW WE DO IT BY HAND (MANUAL)

By hand, we do it like this: First, we do the same as INNER JOIN (match rows that connect). Then, we check: did any employee have no match? Yes, Lex. We add him. Then, we check: did any department have no match? Yes, Marketing and Accounting. We add them. We write "-" where there is no data.

### Our paper (manual result):

| first_name | dept_name |
|---|---|
| Steven | Executive |
| Neena | Executive |
| Lex | - (no dept) |
| Diana | Administration |
| Kimberely | Administration |
| - (no emp) | Marketing |
| - (no emp) | Accounting |

## STEP B: HOW WE WRITE THIS IN SQL

```
SELECT e.first_name, d.dept_name
FROM employees e
FULL OUTER JOIN departments d
ON e.dept_id = d.dept_id;
```

FULL OUTER JOIN = keep all rows from both tables. This is the same as LEFT JOIN + RIGHT JOIN together. Nothing is removed.

| What we did by hand | SQL code that does the same |
|---|---|
| We kept ALL employees | LEFT JOIN part of FULL OUTER |
| We kept ALL departments too | RIGHT JOIN part of FULL OUTER |
| We wrote "-" for empty places | SQL writes NULL |
| Nothing was removed | FULL OUTER JOIN removes nothing |

# Part 6: All 4 JOINs Compared

| Row | INNER | LEFT | RIGHT | FULL OUTER |
|---|---|---|---|---|
| Steven \| Executive | YES | YES | YES | YES |
| Neena \| Executive | YES | YES | YES | YES |
| Lex \| - | NO | YES | NO | YES |
| Diana \| Admin | YES | YES | YES | YES |
| Kimberely \| Admin | YES | YES | YES | YES |
| - \| Marketing | NO | NO | YES | YES |
| - \| Accounting | NO | NO | YES | YES |
| Total rows | 4 | 5 | 6 | 7 |

## Rules (in simple words):

| JOIN Type | What it means (manual) |
|---|---|

INNER JOIN: Match rows. Skip rows with no match. Only matching rows stay.

LEFT JOIN: Match rows. Keep ALL left table rows. Write "-" (NULL) if no match on right.

RIGHT JOIN: Match rows. Keep ALL right table rows. Write "-" (NULL) if no match on left.

FULL OUTER JOIN: Match rows. Keep ALL rows from both tables. Write "-" (NULL) where no match.

## How to remember:

```
INNER = remove everything without a match.
LEFT = protect the left table (employees). Keep them all.
RIGHT = protect the right table (departments). Keep them all.
FULL = protect both tables. Keep everything.
In SQL, the "-" we write by hand is called NULL.
```

## Quick summary of the SQL query:

```
SELECT e.first_name, d.dept_name -- what to show
FROM employees e -- left table (call it e)
[INNER/LEFT/RIGHT/FULL OUTER] JOIN departments d -- right table (call it d) + type
ON e.dept_id = d.dept_id; -- match rule: connect on dept_id
```

## When to use which:

| You want ... | Use ... |
|---|---|
| Only matching rows | INNER JOIN |
| All employees, even with no department | LEFT JOIN |
| All departments, even with no employees | RIGHT JOIN |
| Everything from both tables | FULL OUTER JOIN |

| | |
|---|---|
| Find employees with no department | LEFT JOIN + WHERE d.dept_id IS NULL |